ML Experiment-02

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
df = pd.read_csv("uber.csv")
print("Dataset shape:", df.shape)
display(df.head())
```

```
Dataset shape: (200000, 9)
```

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_la
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.7
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.7
2	44984355	2009-08-24 21:45:00.00000061	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.7
3	25894730	2009-06-26 08:22:21.0000001	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.7
4	17610152	2014-08-28 17:47:00.000000188	16.0	2014-08-28 17:47:00 UTC	-73.925023	40.7

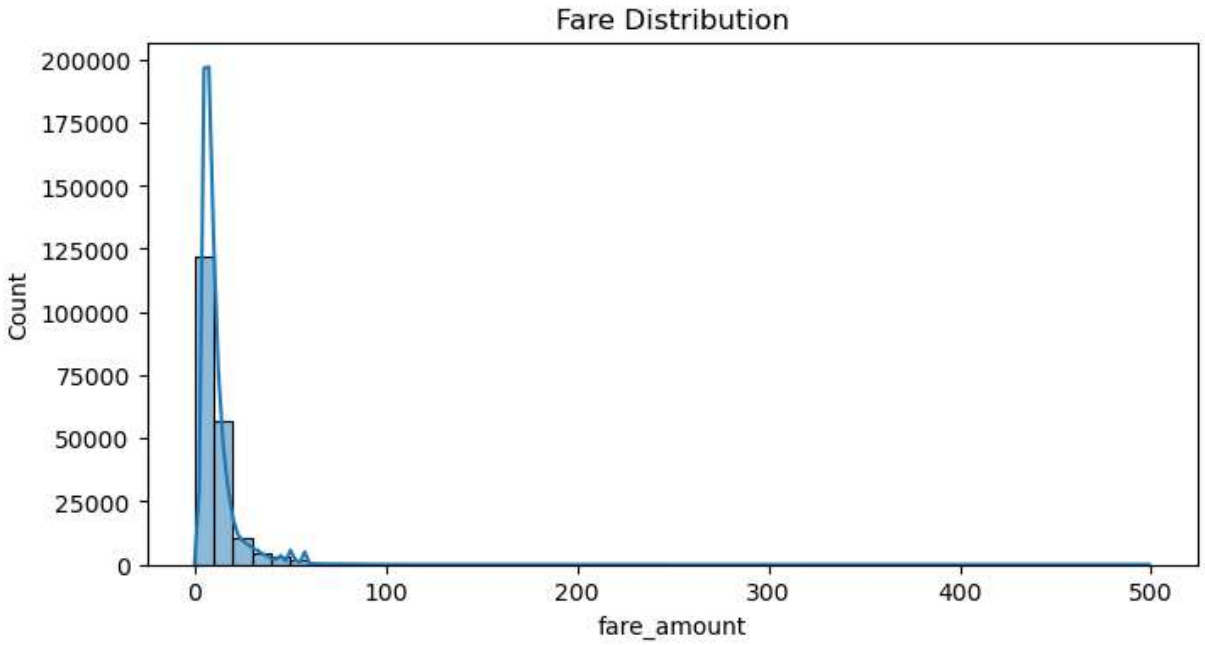
In [2]:
```
df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]
df.drop_duplicates(inplace=True)
numeric_cols = [
    "fare_amount",
    "pickup_longitude",
    "pickup_latitude",
    "dropoff_longitude",
    "dropoff_latitude",
    "passenger_count"
]
for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

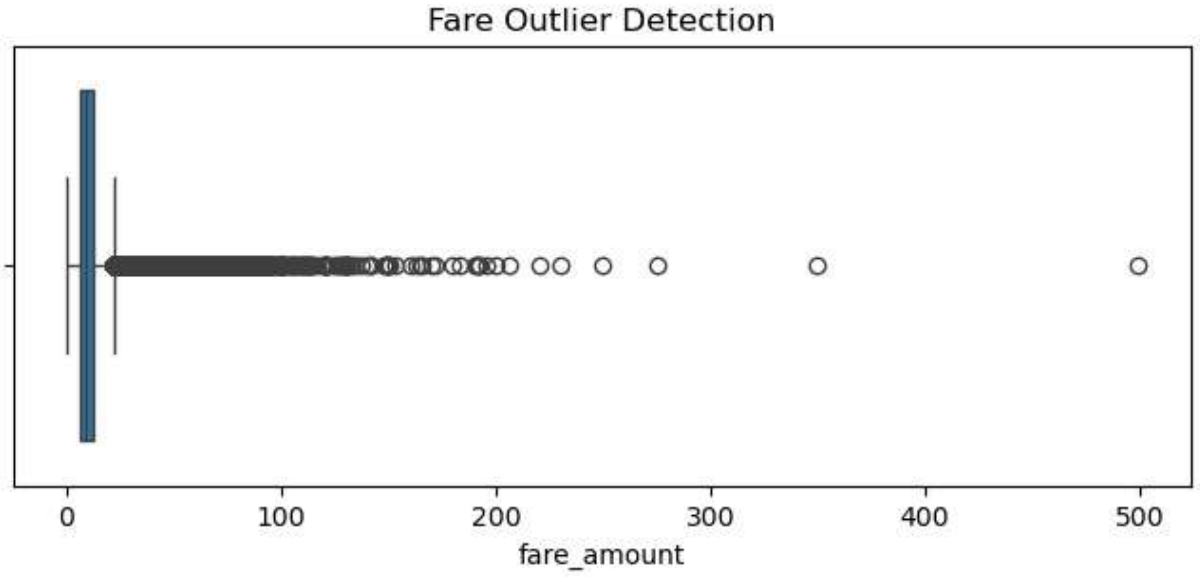
df = df[df["fare_amount"].notna() & (df["fare_amount"] > 0)].copy()
print("Shape after preprocessing:", df.shape)
print(df.isnull().sum())
```

```
Shape after preprocessing: (199978, 9)
unnamed:_0           0
key                  0
fare_amount          0
pickup_datetime      0
pickup_longitude     0
pickup_latitude      0
dropoff_longitude    1
dropoff_latitude     1
passenger_count      0
dtype: int64
```

In [3]:
```
def haversine_distance(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = (np.sin(dlat/2)**2 +
    np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2)
    return 6371 * 2 * np.arcsin(np.sqrt(np.clip(a, 0, 1)))
    coord_cols = ["pickup_longitude","pickup_latitude", "dropoff_longitude","dropof
    if all(c in df.columns for c in coord_cols):
        valid = (df["pickup_longitude"].between(-180,180) &
                 df["dropoff_longitude"].between(-180,180) &
                 df["pickup_latitude"].between(-90,90) &
                 df["dropoff_latitude"].between(-90,90) &
                 (df["pickup_longitude"] != 0) &
                 (df["dropoff_longitude"] != 0) &
                 (df["pickup_latitude"] != 0) &
                 (df["dropoff_latitude"] != 0))
        df = df[valid].copy()
        df["trip_distance_km"] = haversine_distance(
            df["pickup_latitude"], df["pickup_longitude"],
            df["dropoff_latitude"], df["dropoff_longitude"])
        print("Shape after coordinate validation:", df.shape)
        display(df.head())
```

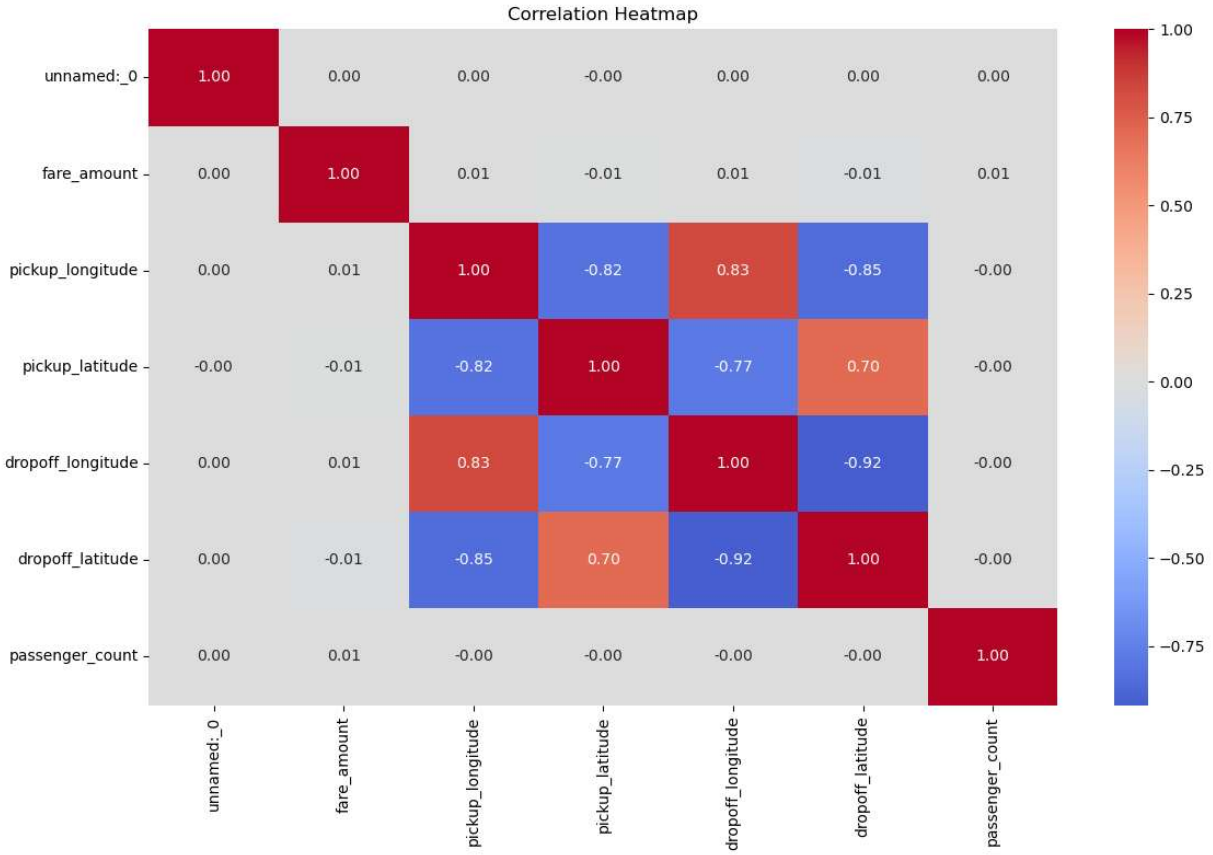
In [4]:
```
plt.figure(figsize=(8,4))
sns.histplot(df["fare_amount"], bins=50, kde=True)
plt.title("Fare Distribution")
plt.show()
plt.figure(figsize=(8,3))
sns.boxplot(x=df["fare_amount"])
plt.title("Fare Outlier Detection")
plt.show()
Q1 = df["fare_amount"].quantile(0.25)
Q3 = df["fare_amount"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR
outliers = df[(df["fare_amount"] < lower) | (df["fare_amount"] > upper)]
print("Q1:",Q1,"Q3:",Q3,"IQR:",IQR)
print("Lower limit:",lower,"Upper limit:",upper)
print("Potential outliers:",len(outliers))
```






```
Q1: 6.0 Q3: 12.5 IQR: 6.5
Lower limit: -3.75 Upper limit: 22.25
Potential outliers: 17156
```

In [5]:
```
numeric_df = df.select_dtypes(include=np.number)
corr = numeric_df.corr()
plt.figure(figsize=(12,8))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f", center=0)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()
print("Correlation with fare_amount:")
display(corr["fare_amount"].drop("fare_amount").sort_values(ascending=False))
```




```
Correlation with fare_amount:
pickup_longitude     0.010467
passenger_count      0.010414
dropoff_longitude    0.009208
unnamed:_0           0.000616
pickup_latitude     -0.008488
dropoff_latitude    -0.011250
Name: fare_amount, dtype: float64
```

In [6]:
```
features = ["pickup_longitude","pickup_latitude",
"dropoff_longitude","dropoff_latitude",
"passenger_count","trip_distance_km"]
features = [c for c in features if c in df.columns]
X = df[features].replace([np.inf,-np.inf],np.nan)
X = X.dropna(axis=1, how="all")
y = df["fare_amount"]
X_train,X_test,y_train,y_test = train_test_split(
X,y,test_size=0.20,random_state=42)
print("Features:",X.columns.tolist())
print("Training samples:",len(X_train))
print("Testing samples:",len(X_test))
```

```
Features: ['pickup_longitude', 'pickup_latitude', 'dropoff_longitude', 'dropoff_lati
tude', 'passenger_count']
Training samples: 159982
Testing samples: 39996
```

In [7]:
```
def make_pipeline(model):
    return Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
```

```
    ("regressor", model)
    ])
models = {
    "Linear Regression": make_pipeline(LinearRegression()),
    "Ridge Regression": make_pipeline(Ridge(alpha=1.0)),
    "Lasso Regression": make_pipeline(Lasso(alpha=0.1, max_iter=20000))
    }
```

In [8]:
```
predictions = {}
for name, model in models.items():
    model.fit(X_train,y_train)
    predictions[name] = model.predict(X_test)
    print(name,"trained successfully")
```

```
Linear Regression trained successfully
Ridge Regression trained successfully
Lasso Regression trained successfully
```

In [9]:
```
results = []
for name,y_pred in predictions.items():
    results.append({
        "Model":name,
        "R2 Score":r2_score(y_test,y_pred),
        "RMSE":np.sqrt(mean_squared_error(y_test,y_pred)),
        "MAE":mean_absolute_error(y_test,y_pred),
        "MSE":mean_squared_error(y_test,y_pred)
        })
    results_df = pd.DataFrame(results).sort_values("RMSE")
    display(results_df.round(4))
    print("Best model based on RMSE:",results_df.iloc[0]["Model"])
```

	Model	R2 Score	RMSE	MAE	MSE
0	Linear Regression	0.0002	9.8253	6.0859	96.5358

```
Best model based on RMSE: Linear Regression
```

	Model	R2 Score	RMSE	MAE	MSE
0	Linear Regression	0.0002	9.8253	6.0859	96.5358
1	Ridge Regression	0.0002	9.8253	6.0859	96.5358

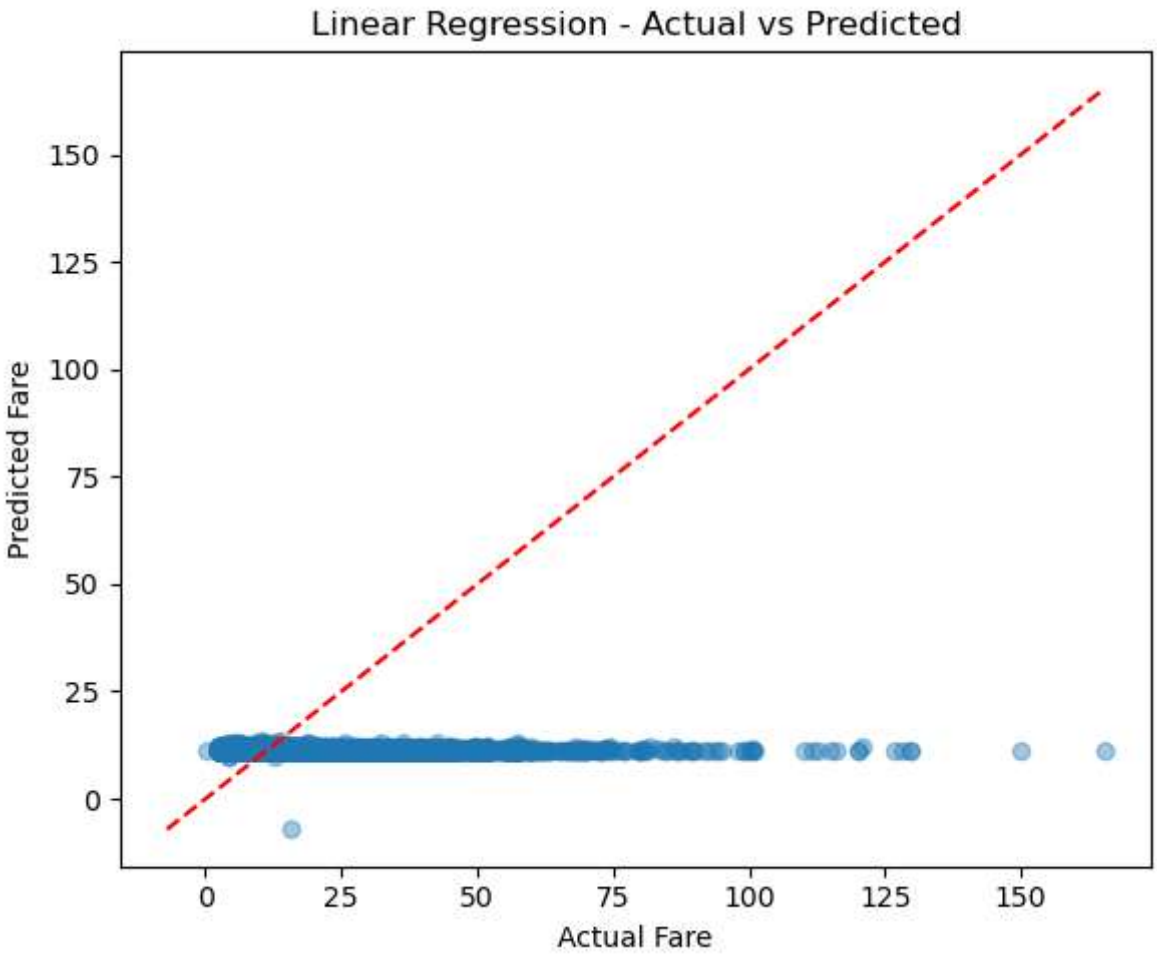
```
Best model based on RMSE: Linear Regression
```

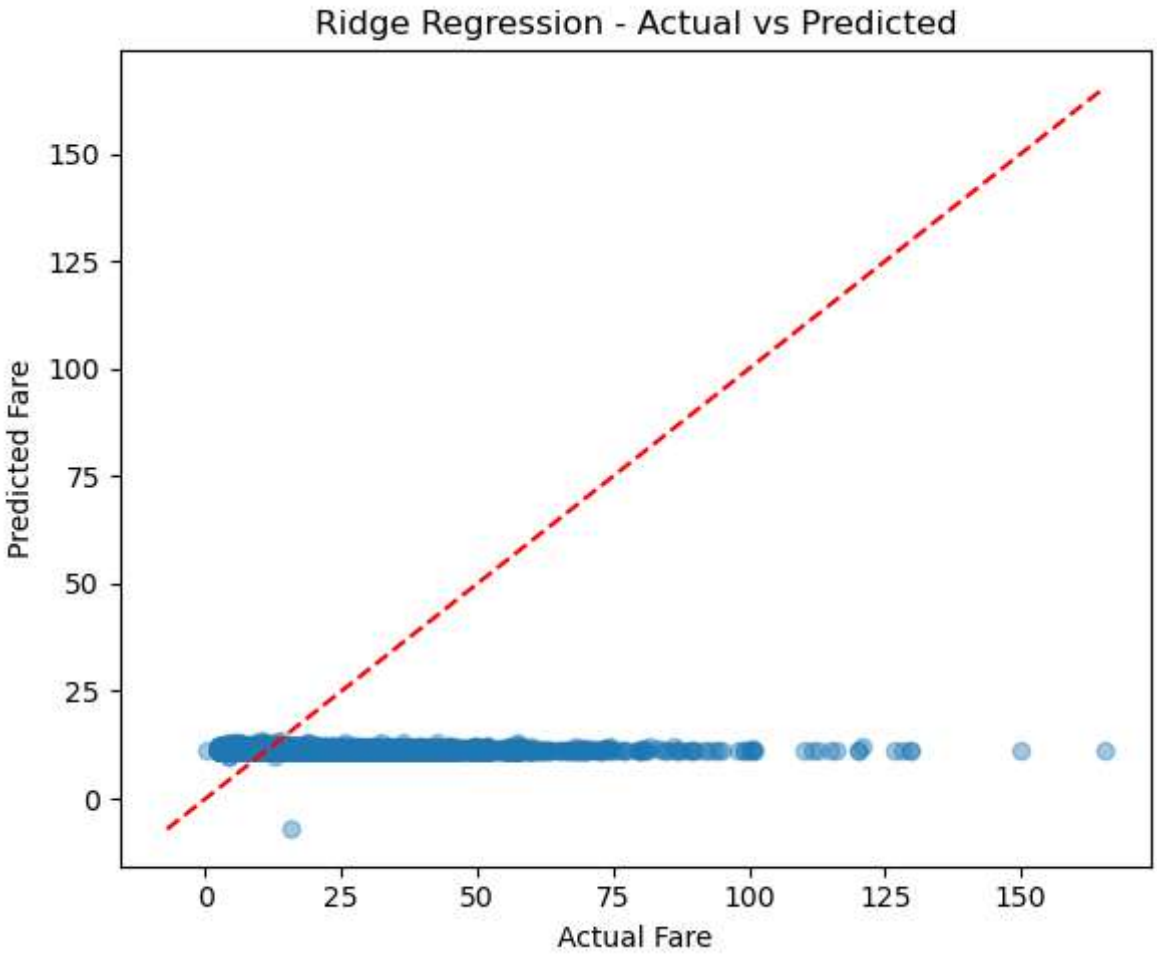
	Model	R2 Score	RMSE	MAE	MSE
0	Linear Regression	0.0002	9.8253	6.0859	96.5358
1	Ridge Regression	0.0002	9.8253	6.0859	96.5358
2	Lasso Regression	-0.0000	9.8265	6.0860	96.5604

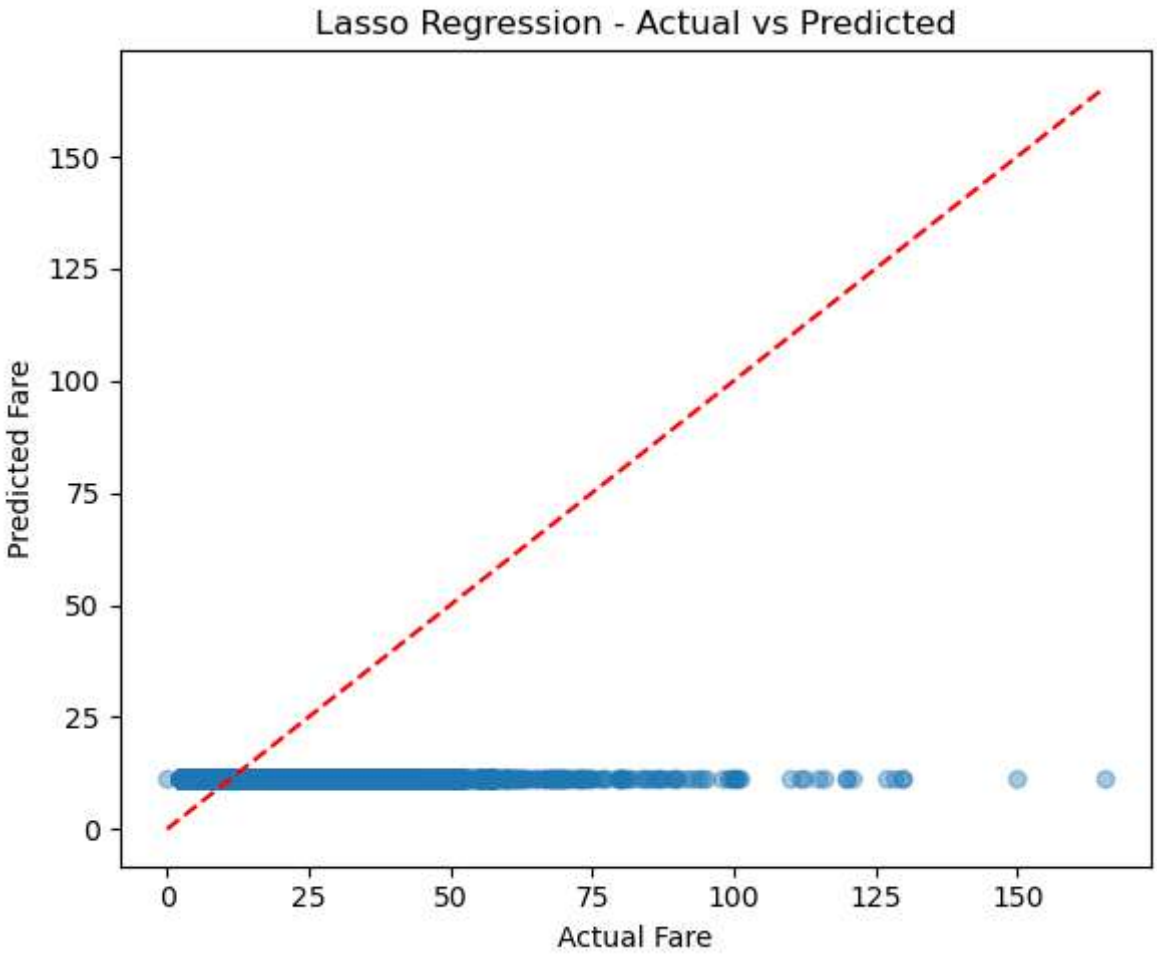
```
Best model based on RMSE: Linear Regression
```

In [10]:
```
for name,y_pred in predictions.items():
    plt.figure(figsize=(6,5))
    plt.scatter(y_test,y_pred,alpha=0.4)
```

```
    mn=min(y_test.min(),y_pred.min())
    mx=max(y_test.max(),y_pred.max())
    plt.plot([mn,mx],[mn,mx],"r--")
    plt.xlabel("Actual Fare")
    plt.ylabel("Predicted Fare")
    plt.title(name+" - Actual vs Predicted")
    plt.tight_layout()
    plt.show()
```




Ridge Regression - Actual vs Predicted
Predicted Fare
Actual Fare
0
25
50
75
100
125
150



In [11]:
```
new_ride = {
    "pickup_longitude":-73.9857,
    "pickup_latitude":40.7484,
    "dropoff_longitude":-73.9851,
    "dropoff_latitude":40.7580,
    "passenger_count":1
}
if "trip_distance_km" in X.columns:
    new_ride["trip_distance_km"] = haversine_distance(
        new_ride["pickup_latitude"],
        new_ride["pickup_longitude"],
        new_ride["dropoff_latitude"],
        new_ride["dropoff_longitude"])
new_ride_df = pd.DataFrame([new_ride]).reindex(
    columns=X.columns,fill_value=0
    )
for name,model in models.items():
    fare = model.predict(new_ride_df)[0]
    print(f"{name}: Predicted fare = {fare:.2f}")
```

```
Linear Regression: Predicted fare = 11.29
Ridge Regression: Predicted fare = 11.29
Lasso Regression: Predicted fare = 11.35
```